

Smart Scheduler

Autonomous LLM-Orchestrated Scheduling via Telegram and Google Calendar Capstone Project Final Report

Executive Summary

Smart Scheduler is an autonomous conversational scheduling engine designed to translate unstructured natural-language chat messages (e.g., from Telegram) into verified Google Calendar events. Traditional monolithic LLM solutions suffer from severe drawbacks: schema hallucinations, date inversion errors, prohibitive token costs from evaluating conversational small talk, and a lack of deterministic conflict control. This project introduces a decomposed three-stage pipeline orchestrated by **n8n**, running on a local, lightweight Windows runtime backed by SQLite and a secure Cloudflare quick tunnel. By decoupling binary intent filtering, entity extraction, and API command synthesis, the system achieves deterministic schema validation, automated self-correction, and conflict-aware scheduling without human calendar-entry overhead.

1. System Architecture & Infrastructure

The architecture is designed for high reliability, minimal resource footprint, and zero vendor lock-in. Unlike complex multi-service container stacks, the modernized production deployment runs directly on a host machine utilizing native Node.js and embedded storage.

Component	Technology	Role & Key Responsibilities
Orchestrator	n8n (v2.x Engine)	Coordinates workflow execution, manages credentials, state persistence, and webhook routing.
Ingress Tunnel	Cloudflare Tunnel (HTTP/2)	Provides persistent HTTPS webhook termination for Telegram Bot API without open firewall ports.
State & Data	Embedded SQLite	Persists credential vault, execution history, and workflow static data (approval queue).
LLM Inference	OpenAI GPT-4o-mini	Zero-shot reasoning with temperature=0 and JSON Mode for structured semantic parsing.
Egress API	Google Calendar API v3	Authenticated via OAuth2 for event queries, collision checks, creation, and updates.

2. Decomposed Three-Stage LLM Pipeline

Instead of relying on a single prompt to parse text, resolve relative dates, and synthesize Calendar API payloads simultaneously, the architecture enforces modular task decomposition across three distinct stages:

Stage 1: Intent Filtering & Temporal Normalization (Early Exit)

- **Gatekeeping:** Acts as an early binary filter (is_scheduling: boolean). Over 80% of conversational chat messages (e.g., greetings, status updates) contain no actionable calendar intent. When non-scheduling messages arrive, the pipeline triggers an **Early Exit**, terminating execution instantly without invoking downstream LLMs or spamming the chat.
- **Temporal Normalization:** Resolves conversational relative expressions (e.g., 'tomorrow after lunch', 'next Monday at 3pm') into absolute ISO 8601 timestamps based on the user's localized reference clock and timezone.

Stage 2: Intent Classification & Entity Extraction

- **Operation Classification:** Categorizes intent into create, modify, or cancel.
- **Entity Extraction:** Extracts structured attributes: event_title, start_datetime, end_datetime (with automatic 60-minute duration inference if omitted), location, and participants.

Stage 3: Calendar API Command Synthesis & Validation

- **Payload Synthesis:** Translates extracted entities into exact Google Calendar Events API v3 specifications (POST for creation, PATCH for updates, DELETE for cancellation).
- **Deterministic JSON Schema Validation:** Output is strictly validated by a downstream JavaScript validator node against Draft-07 JSON Schema rules (validating that end > start, dates conform to ISO 8601, and timezones are valid IANA identifiers).
- **Prompt-Adjusted Retry Loop:** If validation fails, schema error diagnostics are injected back into Stage 3's context: 'Your previous output was rejected due to [errors]. Fix them.' The model self-corrects within 1–2 retries without human intervention.

3. Conflict Detection & Human-in-the-Loop Confirmation

Double-Booking Prevention: Prior to executing event creation or modification, the workflow queries the Google Calendar Events API for active events overlapping the target timeslot. Transparent ('free') and all-day background events are filtered out. If a collision is detected, event insertion is aborted, and the bot replies in Telegram detailing the conflict and calculating the next available free slot of identical duration that day.

Approval Mechanism ('Ask Before Adding'): For private chats or Telegram Business integrations, events can be staged in the workflow's persistent SQLite static data. The bot delivers an interactive card to the owner with inline [■ Add] and [■ Skip] buttons. Upon button interaction, a callback_query is caught, the context is restored, and the calendar transaction completes deterministically.

4. Quantitative Evaluation & Benchmark Results

An automated benchmark protocol (evaluation/evaluate.py) evaluated the proposed three-stage pipeline against an end-to-end single-model baseline across a curated ground-truth dataset of 120 natural-language Telegram messages (covering multi-intent requests, relative temporal offsets, complex attendee lists, and conversational distractors).

Evaluation Metric	Single-Model Baseline	Three-Stage Pipeline (Ours)	Impact / Improvement
-------------------	-----------------------	-----------------------------	----------------------

Intent Accuracy	91.7%	98.3%	+6.6% improvement in intent detection
Entity Extraction (All)	82.5%	95.0%	+12.5% reduction in missing attendees/dates
API Command Validity	86.7%	100.0%	Zero invalid Calendar API calls (due to validator retry)
Non-Scheduling Filter	90.0%	97.5%	Early exit saves ~85% token cost on chit-chat
Mean Latency (E2E)	1.82 s	2.35 s	Acceptable trade-off for 100% schema compliance

5. Key Engineering Findings & Conclusion

- Task Decomposition vs. Monolithic LLMs:** Splitting LLM responsibility into isolated micro-stages yields higher accuracy, eliminates date formatting errors, and makes testing and debugging straightforward.
- Cost Efficiency via Early Exit:** In practical messaging workloads, early exit prevents unnecessary LLM compute on irrelevant conversational noise, keeping operational expenses below \$0.15–\$0.20 per active user per month.
- Deterministic Guardrails:** Probabilistic language models must never be connected directly to external APIs without deterministic intermediaries. Pairing schema validation with automated prompt-adjusted retry guarantees zero schema failures at runtime.
- Standalone Operational Simplicity:** Moving to native n8n on SQLite with Cloudflare HTTP/2 tunneling eliminates container virtualization overhead, enabling robust zero-maintenance deployment on standard desktop workstations.